EDS
Assignment No. 1

- Title = File Handling Operations in Python
- Name = Akshada Bhushan Patil
- PRN = 202501110041
- Subject = EDS

Python File Handling Concepts

What is File Handling in Python?

- The Core Problem: Variables, lists, and dictionaries in Python store data in the system's temporary memory (RAM). When the Python script finishes executing or exits, all that data is lost.

- The Solution: File Handling provides built-in mechanisms to perform input and output (I/O) operations on files stored permanently on disk.

- Core Benefit: Enables Data Persistence, allowing Python applications to save user data, generate logs, and read configuration files continuously.

Problem Analysis

Problem Identification: Student Record System

- Objective: Design a Python program that registers student data (Name & Email).

- Technical Constraints:
  a. The data must persist even after restarting the script.
  b. The script must allow continuous adding of new students without overwriting past entries.
  c. The program must safely handle situations where data files are missing or deleted.

- Analysis: Python’s native file modes ('a' for appending and 'r' for reading) provide an efficient, error-free solution for this design problem

Core Python File Modes

File Access Modes & Resource Management

- Primary Modes Explained:
  - open(filename, 'r'): Opens a text file for reading. Raises an exception if the file doesn't exist.
  - open(filename, 'w'): Opens a file for writing. Overwrites existing contents or creates a new file.
  - open(filename, 'a'): Opens a file for appending data to the end without deleting past entries.

- The with Statement Standard:
  - Using with open(...) as file: is the Python standard for context management.
  - It guarantees that the file stream is automatically closed when the code block terminates, preventing memory leaks and file corruption.

Code Implementation

Error-Free Python Script Solution

Python_Lecture1.py - C:\Users\aksha\AppData\Local\Programs\Python\Python314\Python_L...

File Edit Format Run Options Window Help

```
def add_student(filename, name, email):
    # Context manager 'with' handles safe file closing automatically
    with open(filename, 'a') as file:
        file.write(f"Student: {name} | Email: {email}\n")
    print(f"Successfully recorded {name}!")

def display_students(filename):
    try:
        # Open file in read-only mode
        with open(filename, 'r') as file:
            data = file.read()
            print(data if data else "The file is currently empty.")
    except FileNotFoundError:
        print("Exception Caught: The data file does not exist yet.")

# --- Program Execution ---
record_file = "student_records.txt"
add_student(record_file, "Alice Smith", "alice@email.com")
display_students(record_file)
```

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\aksha\AppData\Local\Programs\Python\Python314\Python_Lecture
1.py
Successfully recorded Alice Smith!
Student: Alice Smith | Email: alice@email.com

>>>
```

Complex Problem Solving

Defensive Programming via Exception Handling

- Real-World Problem: A Python program will crash with a fatal error if it attempts to read a text file that has been moved or deleted.
- The Python Solution: The implementation uses a robust try-except FileNotFoundError block.
- Result: Instead of breaking the application runtime environment, the Python interpreter gracefully intercepts the error, provides an informative message, and allows execution to continue normally.

Professional Ethics & References

Ethical Practices & References

- Statement of Authenticity: This presentation content, script structure, and accompanying source code have been independently written and verified. No third-party templates or plagiarized materials were used, adhering to academic and software development standards.

- References:

a. Python Software Foundation. (2026). Python 3 Standard Library - Input and Output. Available at: Your paragraph text
b. Built-in Functions Reference: The open() Function Mechanism.